PROGRAMACIÓN ORIENTADA A OBJETOS

Desarrollo de una API con Spring Boot

Integrante:

- Gabriel Alejandro García Angulo

Docente:

JOSE DURAN GARCIA

Managua, 13 de Mayo del 2026

1. Descripción del proyecto

El presente proyecto consiste en el desarrollo de una API REST utilizando Spring Boot y Java, cuyo objetivo es gestionar reservas de espacios de estudio dentro de un entorno académico.

La API permite administrar salas de estudio y reservas asociadas a dichas salas, implementando operaciones CRUD para crear, consultar, actualizar y eliminar información. Además, se utiliza una base de datos relacional PostgreSQL para almacenar los datos de forma persistente.

El sistema trabaja con dos entidades principales: SalaEstudio y Reserva. La relación entre ambas entidades es de uno a muchos, ya que una sala de estudio puede tener varias reservas, mientras que cada reserva pertenece únicamente a una sala específica.

2. Tecnologías utilizadas

Para el desarrollo del proyecto se utilizaron las siguientes tecnologías:

Tecnología	Uso dentro del proyecto
Java	Lenguaje de programación principal
Spring Boot	Framework para el desarrollo de la API
Spring Web	Creación de endpoints REST
Spring Data JPA	Persistencia de datos mediante repositorios
Hibernate	Mapeo objeto-relacional entre Java y PostgreSQL

PostgreSQL	Base de datos relacional utilizada
Lombok	Reducción de código repetitivo
Validation	Validación de datos enviados en las peticiones
Maven	Gestión de dependencias del proyecto
Postman	Pruebas de los endpoints de la API
Git y GitHub	Control de versiones y publicación del repositorio

3. Modelo de entidades

El sistema está compuesto principalmente por dos entidades: SalaEstudio y Reserva.

3.1 Entidad SalaEstudio

La entidad SalaEstudio representa una sala disponible para realizar reservas. Esta entidad contiene información básica como el nombre de la sala, su ubicación, capacidad y disponibilidad.

Campos principales:

Campo	Descripción
id	Identificador único de la sala
nombre	Nombre asignado a la sala
ubicacion	Lugar donde se encuentra la sala
capacidad	Cantidad máxima de personas permitidas

disponible	Indica si la sala está disponible

3.2 Entidad Reserva

La entidad Reserva representa una solicitud de uso de una sala de estudio. Contiene los datos del solicitante, la fecha, el horario y el motivo de la reserva.

Campos principales:

Campo	Descripción
id	Identificador único de la reserva
nombreSolicitante	Nombre de la persona que realiza la reserva
correoSolicitante	Correo electrónico del solicitante
fecha	Fecha de la reserva
horaInicio	Hora de inicio de la reserva
horaFin	Hora de finalización de la reserva
motivo	Motivo por el cual se realiza la reserva
salaEstudio	Sala asociada a la reserva

4. Relación entre entidades

El proyecto implementa una relación de uno a muchos entre las entidades SalaEstudio y Reserva.

Una sala de estudio puede tener varias reservas asociadas, pero cada reserva pertenece únicamente a una sala de estudio.

La relación se representa de la siguiente manera:

SalaEstudio 1 ——— * Reserva

En el proyecto, esta relación se implementa mediante JPA/Hibernate usando las anotaciones @OneToMany, @ManyToOne y @JoinColumn.

Esta relación permite consultar las reservas asociadas a una sala específica mediante el endpoint:

GET /api/salas/{id}/reservas

5. Configuración de la base de datos

Para la persistencia de datos se utilizó PostgreSQL. La base de datos creada para el proyecto se llama:

reservas_estudio_db

La conexión con la base de datos se configuró en el archivo application.properties.

Configuración utilizada:

```
spring.application.name=reservas-api
 spring.datasource.url=jdbc:postgresql://localhost:5432/reservas_estudio_db
 spring.datasource.username=postgres
 spring.datasource.password=TU_PASSWORD
```

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.open-in-view=false
server.port=8080
```

La propiedad spring.jpa.hibernate.ddl-auto=update permite que Hibernate cree o actualice automáticamente las tablas necesarias en la base de datos según las entidades definidas en el proyecto.

6. Endpoints implementados

A continuación, se presentan los endpoints principales implementados en la API.

6.1 Endpoints de salas de estudio

Método	Endpoint	Descripción
GET	/api/salas	Lista todas las salas registradas
GET	/api/salas/{id}	Busca una sala por su ID
POST	/api/salas	Crea una nueva sala
PUT	/api/salas/{id}	Actualiza una sala existente

DELETE	/api/salas/{id}	Elimina una sala
GET	/api/salas/{id}/reservas	Lista las reservas asociadas a una sala

6.2 Endpoints de reservas

Método	Endpoint	Descripción
GET	/api/reservas	Lista todas las reservas registradas
GET	/api/reservas/{id}	Busca una reserva por su ID
POST	/api/reservas/sala/{salaId}	Crea una reserva asociada a una sala
PUT	/api/reservas/{id}/sala/{salaId}	Actualiza una reserva
DELETE	/api/reservas/{id}	Elimina una reserva

7. Pruebas realizadas en Postman

Las pruebas de la API fueron realizadas utilizando Postman. Se validaron los endpoints principales para comprobar el funcionamiento de las operaciones CRUD y la relación entre las entidades SalaEstudio y Reserva.

7.1 Crear una sala de estudio

Método utilizado:

POST

Endpoint:

http://localhost:8080/api/salas

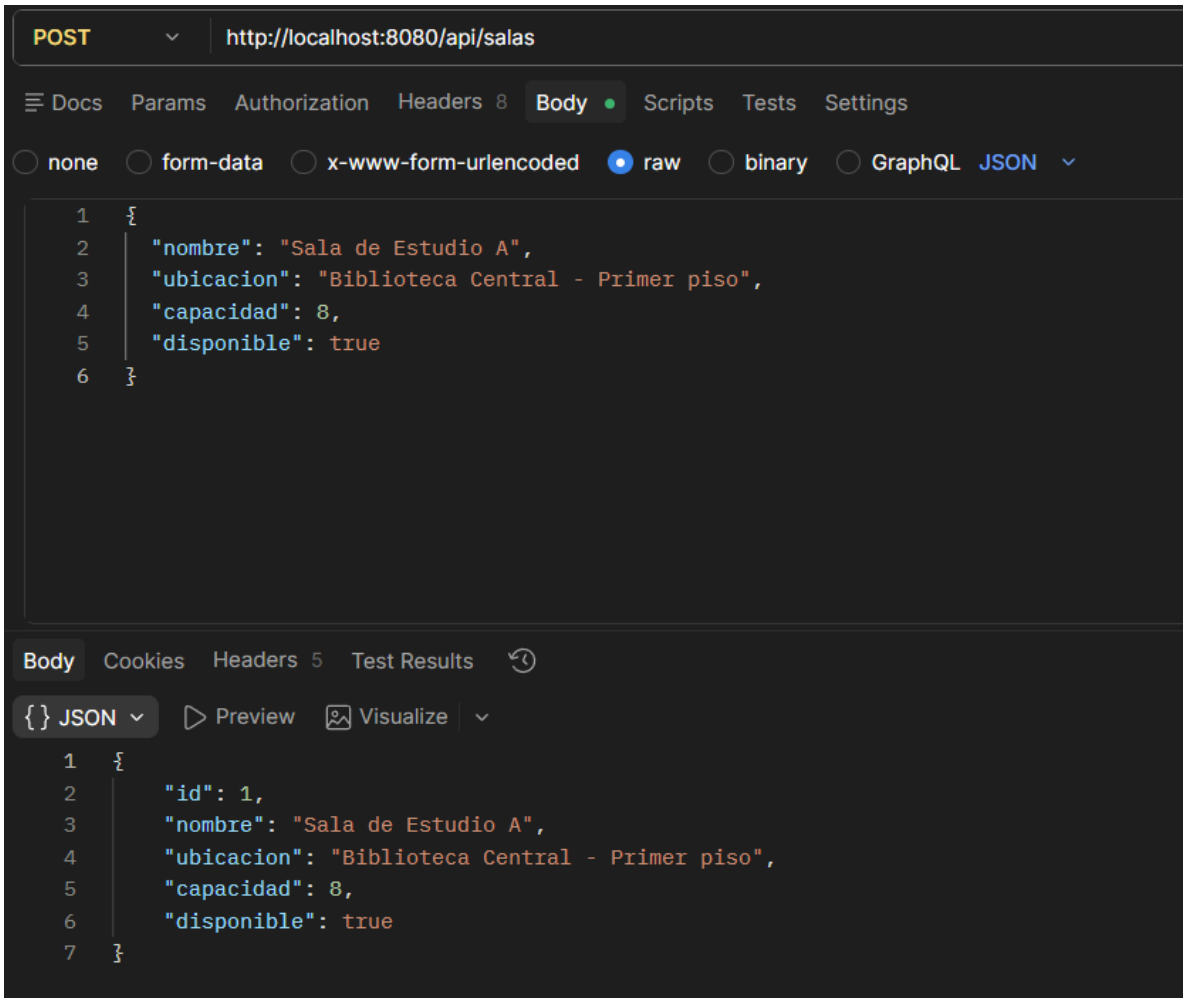
Cuerpo de la petición:

```
{
  "nombre": "Sala de Estudio A",
  "ubicacion": "Biblioteca Central - Primer piso",
  "capacidad": 8,
  "disponible": true
}
```

Resultado esperado:

La API registra una nueva sala de estudio y devuelve la información almacenada junto con su identificador.

Captura de pantalla:



7.2 Listar salas registradas

Método utilizado:

GET

Endpoint:

http://localhost:8080/api/salas

Resultado esperado:

La API muestra una lista con todas las salas registradas en la base de datos.

Captura de pantalla:

GET http://localhost:8080/api/salas

Docs Params Authorization Headers 6 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

1

Body Cookies Headers 5 Test Results

{} JSON Preview Visualize


```
1  [
2      {
3          "id": 1,
4          "nombre": "Sala de Estudio A",
5          "ubicacion": "Biblioteca Central - Primer piso",
6          "capacidad": 8,
7          "disponible": true
8      },
9      {
10         "id": 2,
11         "nombre": "Sala Multimedia",
12         "ubicacion": "Edificio de Ingeniería - Segundo piso",
13         "capacidad": 12,
14         "disponible": true
15     },
16     {
17         "id": 3,
18         "nombre": "Sala de Reuniones B",
19         "ubicacion": "Biblioteca Central - Segundo piso",
20         "capacidad": 6,
21         "disponible": false
22     }
23 ]
```

7.3 Buscar una sala por ID

Método utilizado:

GET

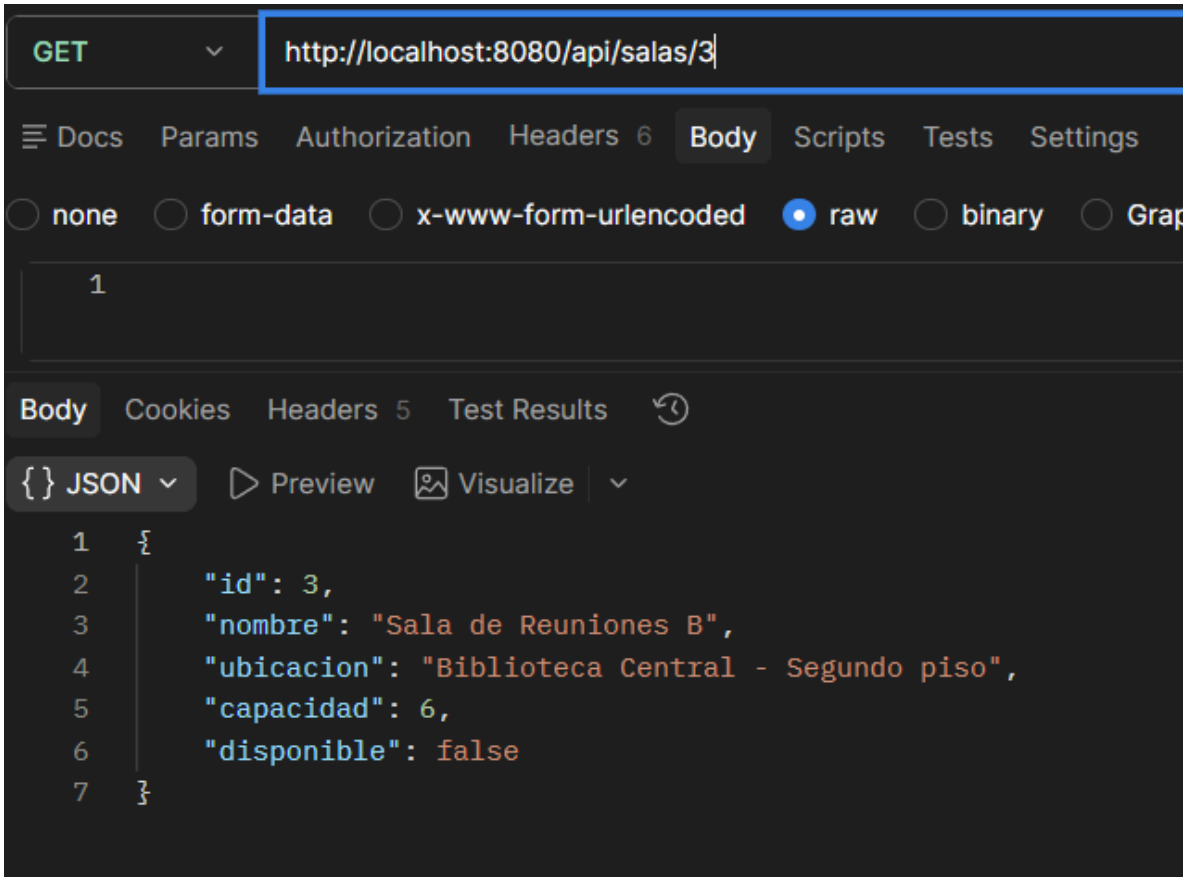
Endpoint:

http://localhost:8080/api/salas/3

Resultado esperado:

La API devuelve la información de la sala correspondiente al ID indicado.

Captura de pantalla:



7.4 Actualizar una sala

Método utilizado:

PUT

Endpoint:

http://localhost:8080/api/salas/1

Cuerpo de la petición:

```
{
  "nombre": "Sala de Estudio A Renovada",
  "ubicacion": "Biblioteca Central - Primer piso",
  "capacidad": 10,
  "disponible": true
}
```

Resultado esperado:

La API actualiza los datos de la sala seleccionada y devuelve la información modificada.

Captura de pantalla:

PUT http://localhost:8080/api/salas/1

Docs Params Authorization Headers 8 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary Gra

```
1 {
2     "nombre": "Sala de Estudio A Renovada",
3     "ubicacion": "Biblioteca Central - Primer piso",
4     "capacidad": 10,
5     "disponible": true
6 }
7
```

Body Cookies Headers 5 Test Results

{} JSON Preview Visualize

```
1 {
2     "id": 1,
3     "nombre": "Sala de Estudio A Renovada",
4     "ubicacion": "Biblioteca Central - Primer piso",
5     "capacidad": 10,
6     "disponible": true
7 }
```

7.5 Crear una reserva asociada a una sala

Método utilizado:

POST

Endpoint:

http://localhost:8080/api/reservas/sala/1

Cuerpo de la petición:


```
{
  "nombreSolicitante": "Gabriel García",
  "correoSolicitante": "garciaga@uamv.edu.ni",
  "fecha": "2026-05-15",
  "horaInicio": "08:00:00",
  "horaFin": "10:00:00",
  "motivo": "Estudio grupal para programación"
 }
```


Resultado esperado:

La API registra una nueva reserva y la asocia automáticamente a la sala con ID 1.

Captura de pantalla:

POST http://localhost:8080/api/reservas/sala/1

Docs Params Authorization Headers 8 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1  {
2    "nombreSolicitante": "Gabriel García",
3    "correoSolicitante": "garciaga@uamv.edu.ni",
4    "fecha": "2026-05-15",
5    "horaInicio": "08:00:00",
6    "horaFin": "10:00:00",
7    "motivo": "Estudio grupal para programación"
8  }
```

Body Cookies Headers 5 Test Results

{} JSON Preview Visualize

```
1  {
2      "id": 1,
3      "nombreSolicitante": "Gabriel García",
4      "correoSolicitante": "garciaga@uamv.edu.ni",
5      "fecha": "2026-05-15",
6      "horaInicio": "08:00:00",
7      "horaFin": "10:00:00",
8      "motivo": "Estudio grupal para programación",
9      "salaEstudio": {
10         "id": 1,
11         "nombre": "Sala de Estudio A Renovada",
12         "ubicacion": "Biblioteca Central - Primer piso",
13         "capacidad": 10,
14         "disponible": true
15     }
16 }
```

7.6 Listar reservas registradas

Método utilizado:

GET

Endpoint:

http://localhost:8080/api/reservas

Resultado esperado:

La API muestra todas las reservas registradas en la base de datos.

Captura de pantalla:

GET http://localhost:8080/api/reservas

Docs Params Authorization Headers 6 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

1

Body Cookies Headers 5 Test Results

{} JSON Preview Visualize


```
1  [
2      {
3          "id": 1,
4          "nombreSolicitante": "Gabriel García",
5          "correoSolicitante": "garciaga@uamv.edu.ni",
6          "fecha": "2026-05-15",
7          "horaInicio": "08:00:00",
8          "horaFin": "10:00:00",
9          "motivo": "Estudio grupal para programación",
10         "salaEstudio": {
11             "id": 1,
12             "nombre": "Sala de Estudio A Renovada",
13             "ubicacion": "Biblioteca Central - Primer piso",
14             "capacidad": 10,
15             "disponible": true
16         }
17     },
18     {
19         "id": 2,
20         "nombreSolicitante": "María López",
21         "correoSolicitante": "maria.lopez@uamv.edu.ni",
22         "fecha": "2026-05-15",
23         "horaInicio": "10:30:00",
24         "horaFin": "12:00:00",
25         "motivo": "Preparación de exposición de POO",
```

GET http://localhost:8080/api/reservas

Docs Params Authorization Headers 6 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JS

1

Body Cookies Headers 5 Test Results

{} JSON Preview Visualize


```
        "motivo": "Preparación de exposición de POO",
        "salaEstudio": {
            "id": 1,
            "nombre": "Sala de Estudio A Renovada",
            "ubicacion": "Biblioteca Central - Primer piso",
            "capacidad": 10,
            "disponible": true
        }
    },
    {
        "id": 3,
        "nombreSolicitante": "Carlos Mendoza",
        "correoSolicitante": "carlos.mendoza@uamv.edu.ni",
        "fecha": "2026-05-16",
        "horaInicio": "13:00:00",
        "horaFin": "15:00:00",
        "motivo": "Reunión para proyecto de sistemas",
        "salaEstudio": {
            "id": 2,
            "nombre": "Sala Multimedia",
            "ubicacion": "Edificio de Ingeniería - Segundo piso",
            "capacidad": 12,
            "disponible": true
        }
    }
```

7.7 Consultar reservas de una sala específica

Método utilizado:

GET

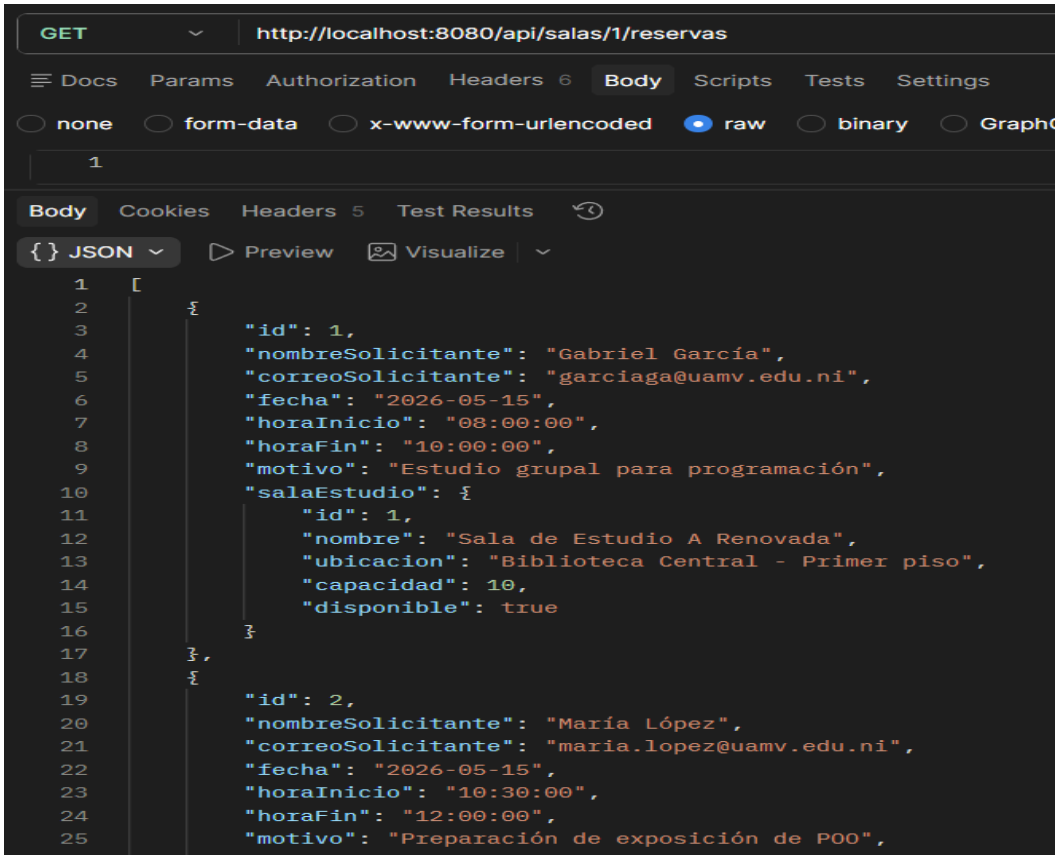
Endpoint:

http://localhost:8080/api/salas/1/reservas

Resultado esperado:

La API muestra únicamente las reservas asociadas a la sala con ID 1, evidenciando la relación entre SalaEstudio y Reserva.

Captura de pantalla:



7.8 Actualizar una reserva

Método utilizado:

PUT

Endpoint:

http://localhost:8080/api/reservas/1/sala/1

Cuerpo de la petición:


```
{
  "nombreSolicitante": "Gabriel García",
  "correoSolicitante": "garciaga@uamv.edu.ni",
  "fecha": "2026-05-15",
  "horaInicio": "08:30:00",
  "horaFin": "10:30:00",
  "motivo": "Estudio grupal actualizado para programación"
}
```


Resultado esperado:

La API actualiza la información de la reserva indicada y mantiene su asociación con la sala correspondiente.

Captura de pantalla:

PUT http://localhost:8080/api/reservas/1/sala/1

Docs Params Authorization Headers 8 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary Graph

```
1  {
2      "nombreSolicitante": "Gabriel García",
3      "correoSolicitante": "garciaga@uamv.edu.ni",
4      "fecha": "2026-05-15",
5      "horaInicio": "08:30:00",
6      "horaFin": "10:30:00",
7      "motivo": "Estudio grupal actualizado para programación"
8  }
9
```

Body Cookies Headers 5 Test Results

{} JSON Preview Visualize

```
1  {
2      "id": 1,
3      "nombreSolicitante": "Gabriel García",
4      "correoSolicitante": "garciaga@uamv.edu.ni",
5      "fecha": "2026-05-15",
6      "horaInicio": "08:30:00",
7      "horaFin": "10:30:00",
8      "motivo": "Estudio grupal actualizado para programación",
9      "salaEstudio": {
10         "id": 1,
11         "nombre": "Sala de Estudio A Renovada",
12         "ubicacion": "Biblioteca Central - Primer piso",
13         "capacidad": 10,
14         "disponible": true
15     }
16 }
```

7.9 Eliminar una reserva

Método utilizado:

DELETE

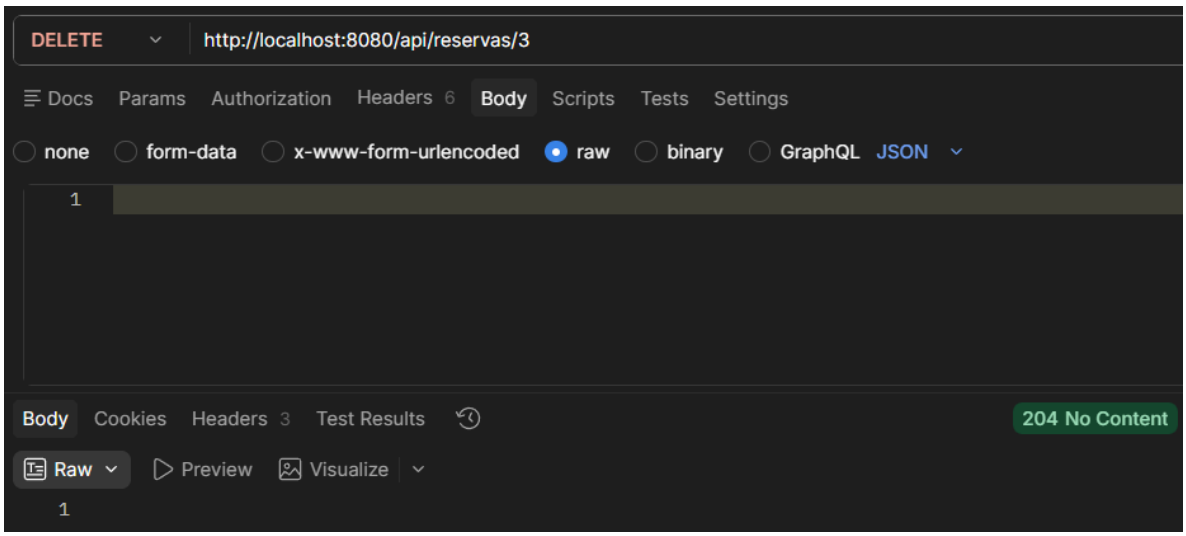
Endpoint:

http://localhost:8080/api/reservas/3

Resultado esperado:

La API elimina la reserva indicada y responde con el estado 204 No Content, confirmando que la eliminación fue exitosa.

Captura de pantalla:


DELETE
http://localhost:8080/api/reservas/3
Docs
Params
Authorization
Headers 6
Body
Scripts
Tests
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
1
Body
Cookies
Headers 3
Test Results
204 No Content
Raw
Preview
Visualize
1

GET http://localhost:8080/api/reservas

Docs Params Authorization Headers 6 Body Scripts Tests Settings

Body Cookies Headers 5 Test Results

{} JSON Preview Visualize


```
[
    {
        "id": 2,
        "nombreSolicitante": "María López",
        "correoSolicitante": "maria.lopez@uamv.edu.ni",
        "fecha": "2026-05-15",
        "horaInicio": "10:30:00",
        "horaFin": "12:00:00",
        "motivo": "Preparación de exposición de POO",
        "salaEstudio": {
            "id": 1,
            "nombre": "Sala de Estudio A Renovada",
            "ubicacion": "Biblioteca Central - Primer piso",
            "capacidad": 10,
            "disponible": true
        }
    },
    {
        "id": 1,
        "nombreSolicitante": "Gabriel García",
        "correoSolicitante": "garciaga@uamv.edu.ni",
        "fecha": "2026-05-15",
        "horaInicio": "08:30:00",
        "horaFin": "10:30:00",
        "motivo": "Estudio grupal actualizado para programación",
        "salaEstudio": {
            "id": 1,
            "nombre": "Sala de Estudio A Renovada",
            "ubicacion": "Biblioteca Central - Primer piso",
```

8. Evidencia de funcionamiento

Con las pruebas realizadas en Postman se comprobó que la API permite:

- Crear registros en la base de datos.
- Consultar registros individuales y listados generales.
- Actualizar información existente.
- Eliminar registros.
- Asociar reservas a salas de estudio.
- Consultar las reservas pertenecientes a una sala específica.

Estas pruebas evidencian el funcionamiento de los endpoints principales y el cumplimiento de las operaciones CRUD solicitadas.

9. Enlace del repositorio

El código fuente completo del proyecto se encuentra disponible en el siguiente repositorio público de GitHub:

https://github.com/THEGABOALE/reservas-api

10. Conclusión

El proyecto desarrollado cumple con los requisitos establecidos, ya que implementa una API funcional utilizando Spring Boot, Java y PostgreSQL. Además, se aplicaron operaciones CRUD completas, una relación entre entidades mediante JPA/Hibernate y pruebas de funcionamiento utilizando Postman.

La estructura del proyecto se organizó en capas, separando modelos, repositorios, servicios y controladores, lo que permite mantener un código más claro, ordenado y fácil de mantener.